

3-Mark Questions & Answers — Data Structures Module 2: Stacks & Queues

Q1. Explain the algorithm for checking Well-Formedness of Parentheses using a Stack.

1. Scan the arithmetic string expression from left to right.
2. If an opening bracket (`(` , `{` , `[`) is encountered, **push** it onto the stack.
3. If a closing bracket (`)` , `}` , `]`) is encountered:
 - If stack is empty: **Unbalanced** (Excess closing bracket).
 - **pop** top element; if popped bracket does not match closing bracket: **Unbalanced** (Mismatched type).
4. After scanning, if stack is empty -> **Balanced/Well-Formed**; else **Unbalanced**.

Q2. Differentiate between Infix, Prefix, and Postfix notations.

Notation	Operator Position	Example	Evaluation
Infix	Operator between operands.	<code>(A + B) * C</code>	Requires operator precedence and parentheses.
Prefix (Polish)	Operator before operands.	<code>* + A B C</code>	Evaluated from right to left without parentheses.
Postfix (Reverse Polish)	Operator after operands.	<code>A B + C *</code>	Evaluated from left to right using a Stack.

Q3. Explain Priority Queue and its types.

A **Priority Queue** is a queue where each element has an assigned priority:

- **Ascending Priority Queue:** Element with smallest numerical value has highest priority (served first).
- **Descending Priority Queue:** Element with largest numerical value has highest priority.